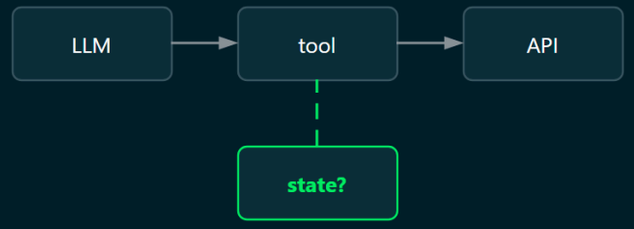
What Should a Local Agent Remember?

Saving state for agents running at the edge

By the end you will be able to: 1 · model and index the state of an agent workflow as documents the agent can read in one go 2 · decide what the agent remembers: what reacts to change, what expires, and what it must be able to explain



Eoin Jordan · 40 minutes · live on a RUBIK Pi 3 (Qualcomm QCS6490) · Ubuntu 24.04 · MongoDB 9.0 Community

After this session you will be able to

1

Model and index the state of a workflow

as documents the agent can read in one go, with the index each question needs

2

Decide what the agent remembers

what reacts to change, what expires, and what it must be able to explain

For experienced engineers who know documents, indexes and change streams, and have not yet built an agent that has to survive a restart.

Eoin Jordan

Senior Developer Relations Engineer, Edge Impulse / Qualcomm · eight years at IBM Galway on distributed systems · PhD, Insight Centre, University of Galway · MongoDBSimCity on the side
MongoDB skill badges, October 2026: Building AI Agents with MongoDB · AI and Innovation: How MongoDB Enables a Resilient AI Strategy · Schema Design Patterns (in progress)

Follow the database side live

MongoDBSimCity: walk through a MongoDB 9.0 server in your browser while we talk

eoinjordan.github.io/MongoDBSimCity

Before we start: three questions for you

1 **Are you running an agent in production?**

- Yes
- Piloting
- Not yet

2 **Where does your agent's state live today?**

- In the prompt
- Files or cache
- A database
- Nowhere, honestly

3 **What has hurt most?**

- Lost IDs / orphaned jobs
- Polling and latency
- Token cost
- Nobody can explain what it did

Zoom poll is open

Or type 1 / 2 / 3 with your answer in the chat, e.g. “2: prompt”.

`2: prompt`

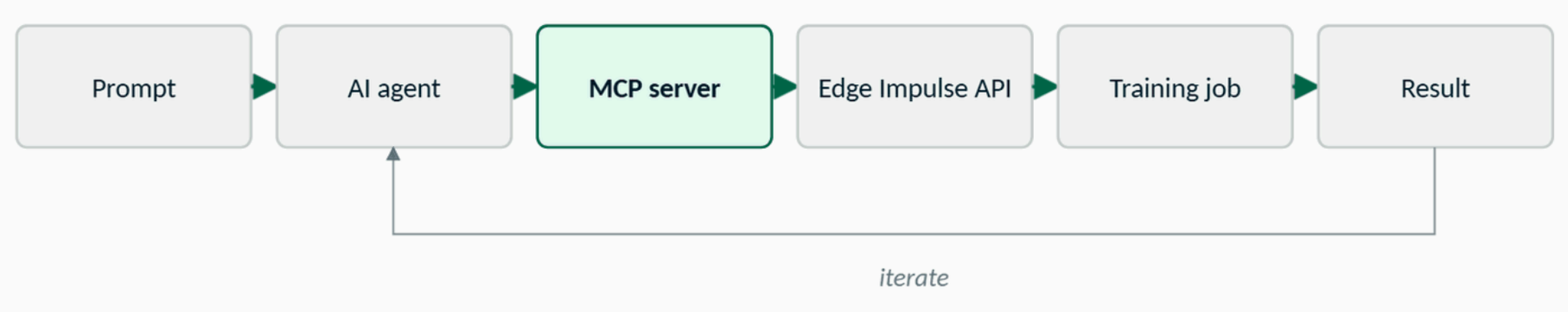
`3: polling`

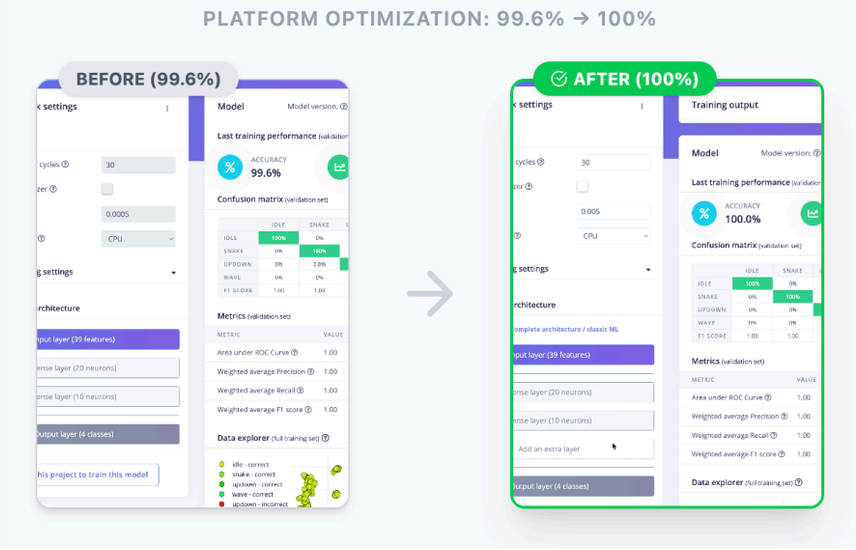
We come back to your answers twice during the talk.

What I actually built

The write-up

A natural-language control plane over Edge Impulse Studio





"Train this project, optimise for accuracy under constraints, and tell me what changed."

What came back: the same project, retrained by the agent under those constraints, 99.6% to 100%, with a summary of what it changed. (Screenshot from the blog post.)

How I stopped it bluffing

Three moves, before any database was involved:

1

Stub the whole API

mock first, real jobs later

2

Test every tool

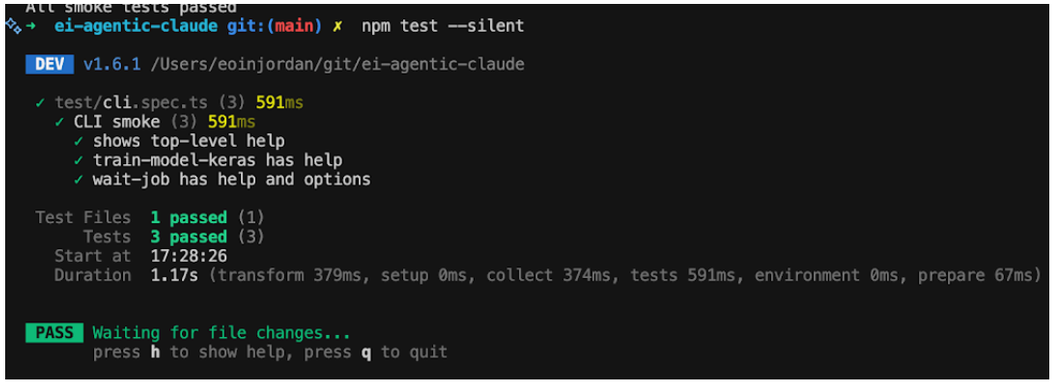
happy · failure · missing params · edge cases

3

Add agent-friendly helpers

wait / status / retry · scoping · validation

Remember helper 3: it's the second half of this talk.

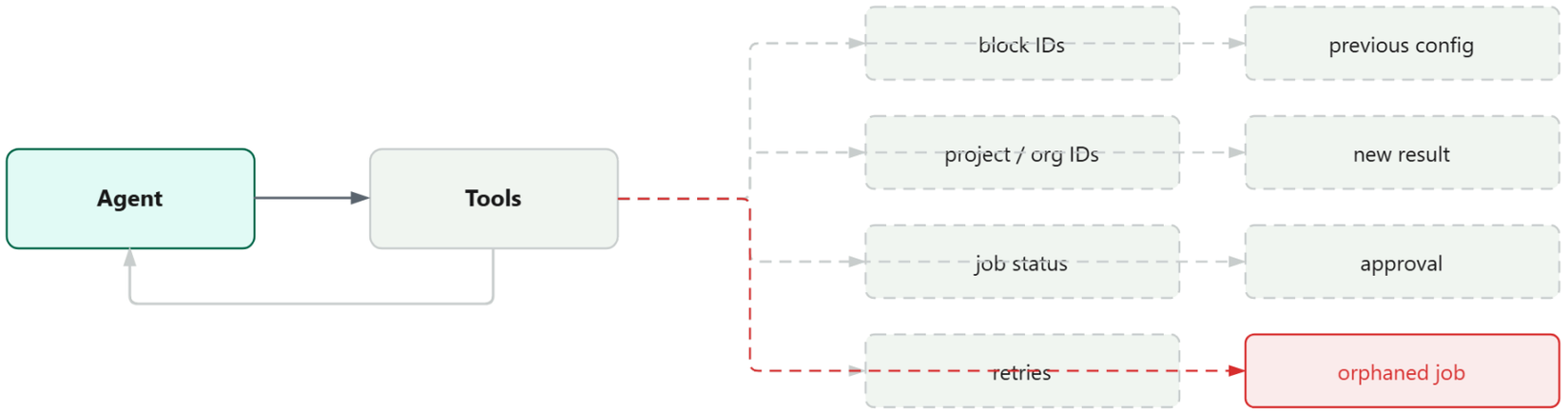


The per-tool test run from the blog post: happy path, failure path, missing params, edge cases.

The key wasn't MCP. It was discipline so the agent couldn't bluff its way through.

Then it started leaking state

IDs everywhere
Job timing
State management



IDs everywhere · job timing · state management

None of this is an LLM problem.
It is a state problem.

Where does this live?

In the prompt?

In a file?

In a cache?

In a database?

Today we fix that: two objectives

You already know documents, indexes and change streams. New today: using them as an agent's memory.

1

Model and index the state

of an agent workflow as documents the agent can read in one go, with the index each question needs

2

Decide what memory means

short-term versus long-term: what reacts to change, what expires, what must be explainable later

Three parts: the document and its indexes · reacting to change · what to keep, what to expire, what to explain

Each idea is applied to the same running example, a training run.

Start with the questions

What will the system ask when things go wrong?

What runs are active for this project?

What changed since the last good run?

Which jobs are still pending?

Which actions were approved by a human?

What previous failure looks like this one?

Decide the questions first. The document's shape follows them.

Relationships, read patterns and write patterns decide what goes where.

Our first document

live on the RUBIK Pi

Data modelling docs

```
{
  runId:      "run-1842",
  projectId:  "project-512482",
  orgId:      "org-71",
  objective:  { metric: "accuracy", target: 0.92 },
  status:     "training",
  configuration: { windowMs: 2000,
                   learningRate: 0.001 },
  jobs: [
    { jobId: "job-203", type: "training",
      status: "running", startedAt: ... }
  ],
  results:    { accuracy: 0.89, latencyMs: 14 },
  approvals: [
    { action: "train", by: "human", at: ... }
  ],
  createdAt:  ..., updatedAt: ...
}
```

What's wrong with it?

What grows without bound?

What changes every few seconds?

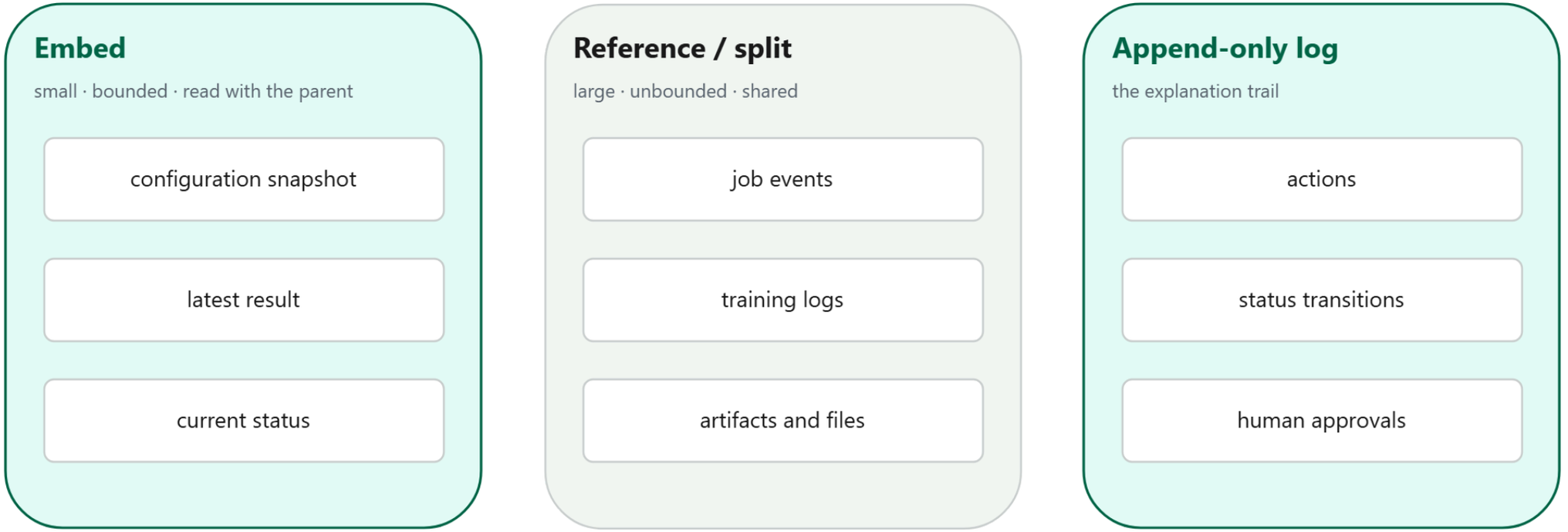
What needs a history, not a field?

Which query must be fast?

What should not be in here at all?

A useful first model to challenge.

Embed, reference, or log?



Decide by change, growth and access. The reasoning matters more than any rule of thumb.

The question creates the index

What the agent asks

```
runs.find({ projectId: "project-512482",
            status: "training" })
```

The index that answers it

```
{ projectId: 1, status: 1 }
```

```
runs.find({ projectId: "project-512482" })
    .sort({ updatedAt: -1 }).limit(1)
```

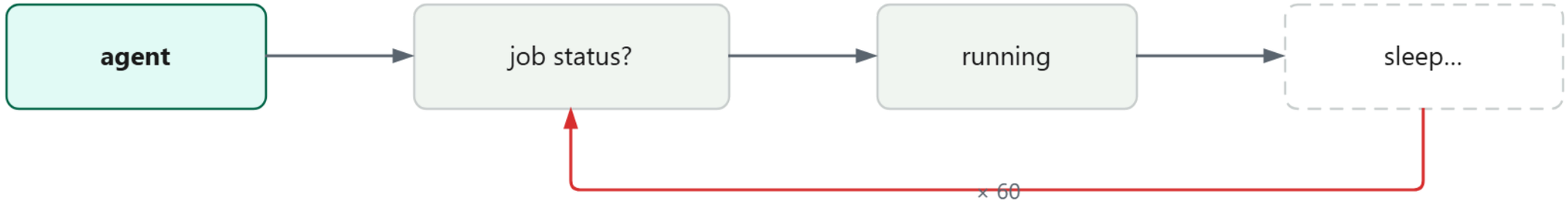
```
{ projectId: 1, updatedAt: -1 }
```

Reads	Writes	Storage
faster	**slower**	**larger**

Equality, then sort, then range. An index is a prediction about how the system will be used. Build them for the questions you actually ask.

Then the wait loop hurt

The “train + wait + summarise” skill, as actually implemented



every check is a read · every wait is latency and tokens · every restart is a place to lose the job ID

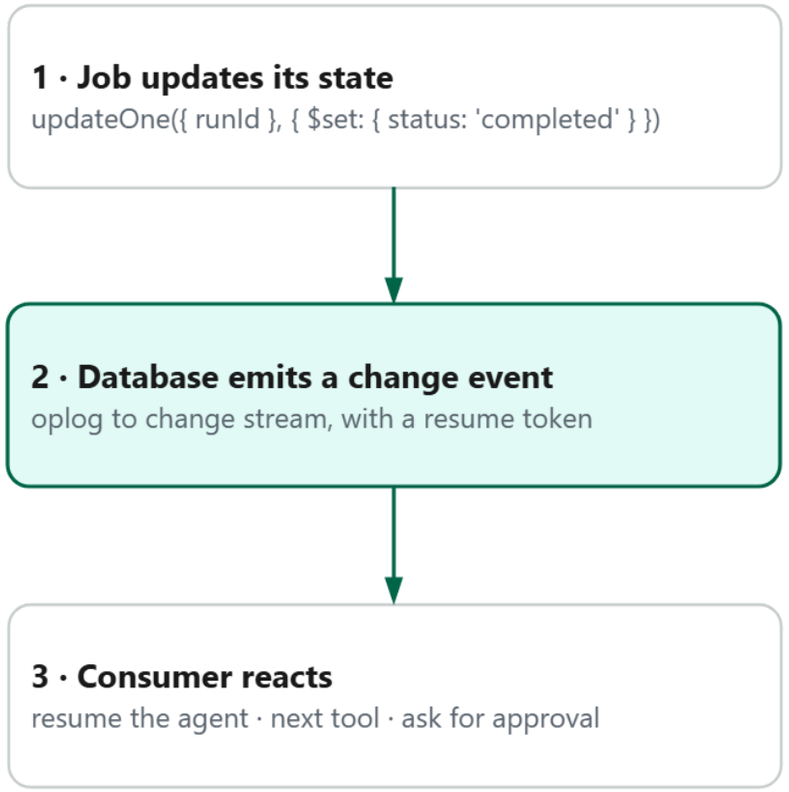
Every check is a read. Every wait is latency and tokens. Every restart is a place to lose the job ID.

The agent was doing the database's job..

From polling to events

live on the RUBIK Pi

Change streams docs




```
pipeline = [{"$match": {
    "operationType": "update",
    "updateDescription.updatedFields.status":
        {"$in": ["completed", "failed"]}
}}]

with runs.watch(pipeline,
                full_document="updateLookup") as cs:
    for change in cs:
        run = change["fullDocument"]
        agent.resume(thread_id=run["runId"])
```

Still design the failure path: missed events, retries, idempotency, resume.

No polling loop. No lost job IDs. The event carries the document the agent needs next.

Memory has a half-life

TTL index docs

Keep	Expire	Review
audit history	temporary job state	raw logs
experiment summaries	session context	failed attempts
approvals	cache	large prompts
baseline results	short-lived tool outputs	intermediate artifacts

```
db.sessions.createIndex(
  { expiresAt: 1 }, { expireAfterSeconds: 0 })
// the writer sets expiresAt per document
```

Lifetime becomes a schema decision, set when the document is written, not a cron job.

Short-term memory expires; long-term memory is kept and audited. TTL enforces the first. It never decides which is which.

What would you delete?

Everything the agent wrote this week, in one list. Which of these has to live?

Audit history · Experiment summaries · Approvals
Session context · Raw logs · Failed attempts · Cache

Not every question is semantic

Vector Search docs

Failed jobs for project X, today.

```
runs.find({
projectId: "project-512482",
"jobs.status": "failed",
updatedAt: { $gte: today } })
```

Previous failures like this one.

```
runs.aggregate([{ $vectorSearch: {
path: "failureEmbedding",
queryVector: embed(thisFailure),
filter: { projectId: ... } }}])
```

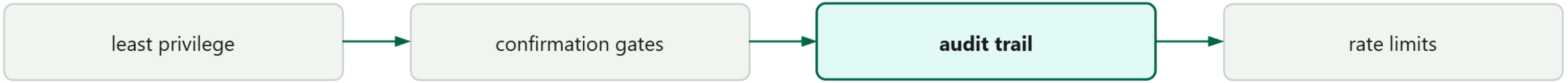


The same collection and the same document answer a different kind of question.

A reliable agent needs both: exact operational state, and similarity over messy context, governed by the same rules.

An agent that can act must be explainable

IDC: the control point



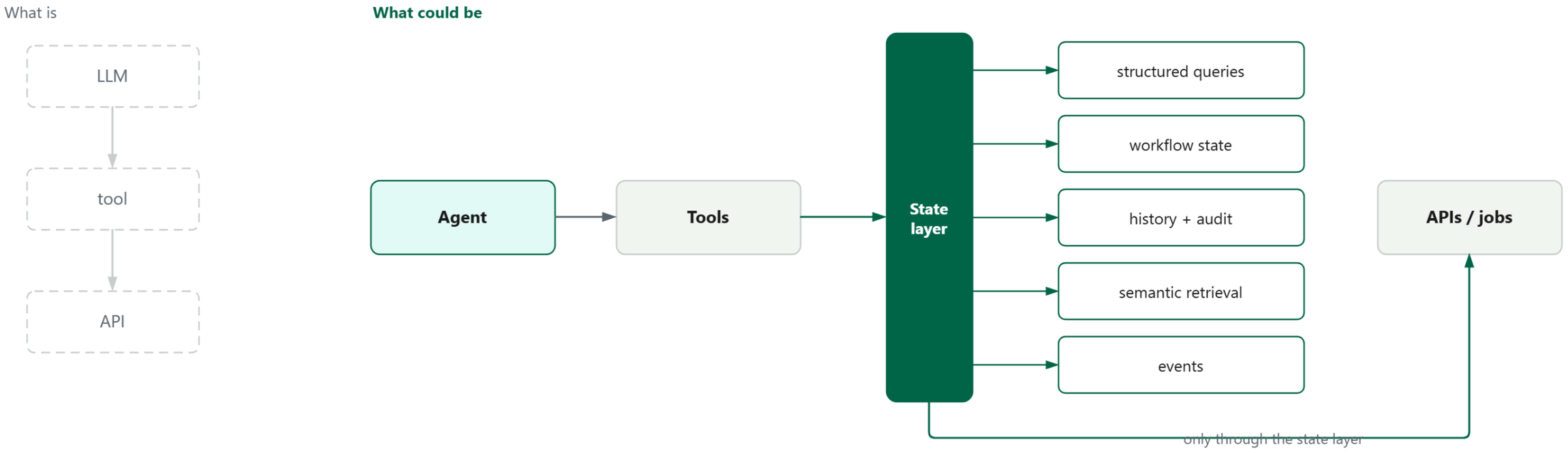

```
{
  action:      "start_training",
  requestedBy: "agent",
  approvedBy:  "eoin",
  scope:       { orgId: "org-71",
                 projectId: "project-512482" },
  decision:    "approved",
  reason:      "non-destructive training run",
  at:          ISODate("...")
}
```

If a system can act, it must be able to explain how that action was allowed.

Approvals and audit live in collections with a schema. The database checks the policy before the action fires, so a bad model output cannot become a bad action.

Guardrails are data.

The resolved architecture



Giving an agent tools is easy. Giving it reliable state is the hard part.

What to take back

One Monday-morning action for each of you:

Developer

Give every workflow a document and a thread ID.

Architect

Put a state layer between tools and the world.

Data / platform lead

Index from the query log; make lifetimes a schema decision with TTL.

Product / engineering manager

Ask one question of any agent project: can it explain what it did?

Back to your chat answers: which of these is yours?

Three things I want you to remember

Model for the workload.

Index for the questions.

Treat AI memory as data.

Where to go next

Routed by what you need next, one QR each:

Learn

Badges and books

- Skill Badges: Building AI Agents with MongoDB, Schema Design Patterns
- Book: From Systems of Record to Agentic Systems of Action (Packt, 2026)
- The Official MongoDB Guide

learn.mongodb.com/skills · mongodb.com/books

Ask

Answered directly

- Community Forums: engineers answer
- Docs, and the docs assistant
- The four doc pages behind this talk (corner QRs)

mongodb.com/community/forums

Build

Give your coding agent the skills

- MongoDB Agent Skills: schema design, query optimizer, MCP setup
- npx skills add mongodb/agent-skills
- Bundled in the Claude, Codex, Cursor and Gemini plugins

mongodb.com/docs/agent-skills

Scale

Customers and partners

- Your account team
- Or connect with an expert
- Bring the run document: it's the conversation starter

mongodb.com/company/contact

Thank you

Questions. I can screen-share the RUBIK Pi demo for any of this.

Code

github.com/eoinjordan
linkedin.com/in/eoin-jordan

The model

eoinjordan.github.io/MongoDBSim

The write-up

Lessons learned building an agentic MCP server (Edge Impulse blog)

A 10-minute lab from this talk

1. Given a messy agent workflow log, identify the state that must persist.
2. Create a first run document and argue what to embed, reference or log.
3. Write two query patterns and derive one compound index, then name its write cost.
4. Replace one polling loop with a change-stream consumer that survives a restart.
5. Add TTL for temporary session data and keep the audit history.

Build the design habit on a problem the learner has already felt.

Agent memory is data: short-term and long-term

LangGraph integration

Short-term

State between steps

Checkpoint per node transition, keyed by the run. Survives a restart. Pauses for human approval.

```
MongoDBSaver(client)
thread_id = run["runId"]
interrupt_before=["tools"]
```

Long-term

What the agent learned across runs

Memories namespaced by project. Looked up by key, or semantically with a vector index.

```
MongoDBStore(...)
namespace = ("project-512482",
             "experiments")
```

The helpers I bolted on in five evenings are what these libraries now do for you, on top of the same collections.

When documents aren't enough

Documents

Records and workflow state

"What is the status of run 1842?"

the operational truth: fast, indexed, audited

Vectors

Similarity over messy context

"Which failures look like this one?"

a query mode beside the record, filtered by it

Graph / RDF

Relationships and inference

"Which devices are affected by this dataset change?"

$graphLookup for bounded hops; a triple store when you need reasoning

Pick the store by the shape of the question, not the shape of the data.

Answering off-script

1

Say it back

one sentence, so you're answering the question they asked

2

Name the mechanism

“this is really a question about where the write cost lands”

3

Walk it on the example

two or three steps: a run, a job, a status change

4

Say what you'd check

what you'd measure before committing, and where you're unsure

They are scoring the method, not the answer. Reasoning aloud is the deliverable.

The helpers I wrote by hand now ship

What I built in five evenings	What MongoDB ships as Agent Skills	
Stub the whole API	**MongoDB MCP Server**	the agent talks to the real database: real schemas, real indexes, no guessing
Test every tool	**Query Optimizer skill**	checks the index serves the query; finds the equality-sort-range order
Agent-friendly helpers	**Schema Design + NL Querying skills**	embed vs reference, validation rules, natural language to aggregation

```
npx skills add mongodb/agent-skills   ·   /schema-design
/query-optimizer
```

Skills are the curriculum, written for the agent. Bundled with the MCP Server in the Claude, Codex, Cursor and Gemini plugins.

Likely questions

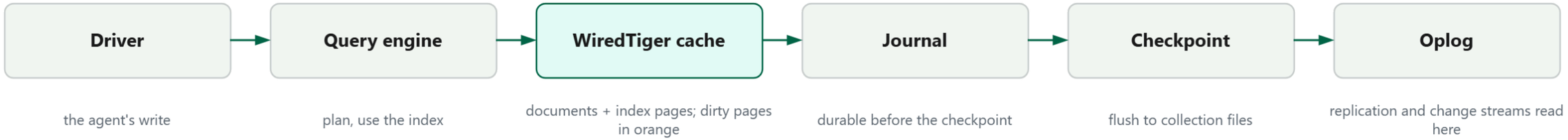
Schema	What if jobs grow without bound? When would you split runs / jobs / events? The 16 MB limit and the subset pattern.
Indexing	Why this compound order? What write cost is acceptable for a status field that changes every few seconds? Covered queries.
Change streams	What happens when the consumer is down? Resume tokens, at-least-once delivery, idempotent handlers.
Lifecycle	TTL runs roughly once a minute: what does that mean for correctness? What never gets a TTL?
Vectors	What to embed, what to filter on metadata, how to evaluate retrieval. Why not a separate vector store?
Safety	How to represent approvals, scopes and destructive actions. Transactions for the approval step.

If runs outgrow one node: sharding

`runId`	Unique, spreads evenly, but every project query becomes scatter-gather. Wrong key for our questions.
`projectId`	Targets both hot queries. Risk: one very busy project is a hot shard.
`orgId`	Even coarser. Fine for isolation, worse for balance.
`{ projectId, createdAt }`	Targeted and time-ordered within a project; the usual compromise for append-heavy workflows.
`hashed projectId`	Best spread, loses range-by-time. Choose when balance matters more than recency scans.

Shard by what you query by. Change streams still work across shards; ordering is per-shard.

What the server is doing underneath



Why it matters for the agent

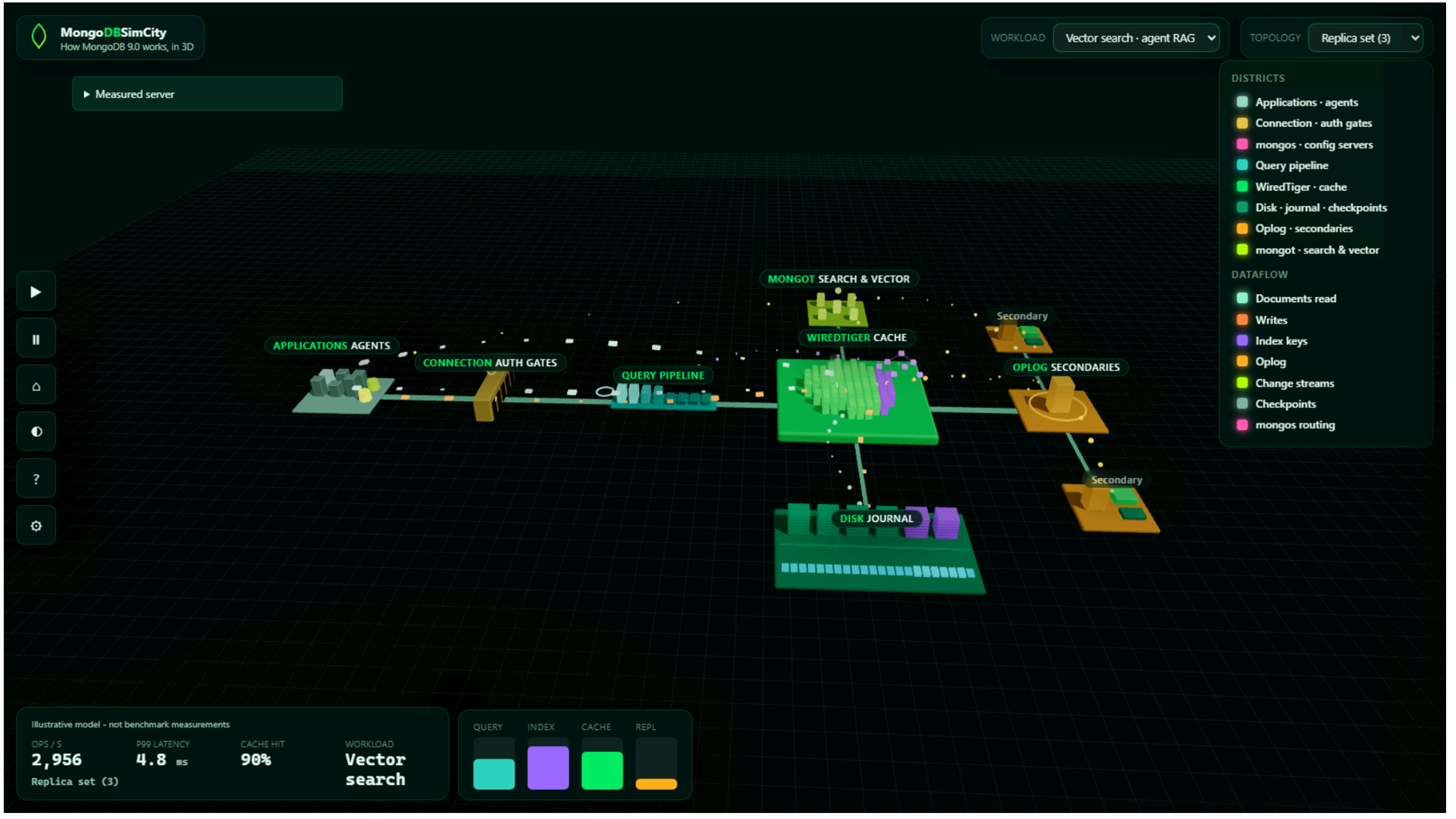
A status write every few seconds dirties a page every few seconds. Unbounded arrays make that page large and the eviction (80% target, 95% trigger) more expensive.

Every index is pages in the same cache as the documents, so the write cost on the index slide is cache pressure, not just disk.

Change streams read the oplog, so the consumer is never on the write path.

Live view: eoinjordan.github.io/MongoDBSimCity: illustrative numbers, documented thresholds.

Backup: the database side, live (MongoDBSimCity)



Sources

Lessons learned building an agentic MCP server	edgeimpulse.com/blog/lessons-learned-from-building-an-agentic-ai-mcp-server-for-edge-impulse/
MongoDB data modelling	mongodb.com/docs/manual/data-modeling/
Change streams	mongodb.com/docs/manual/changeStreams/
TTL indexes	mongodb.com/docs/manual/core/index-ttl/
Atlas Vector Search overview	mongodb.com/docs/atlas/atlas-vector-search/vector-search-overview/
LangGraph + MongoDB (checkpointer, store)	mongodb.com/docs/atlas/ai-integrations/langgraph/
IDC: Your database just became your AI control point	mongodb.com/company/blog/news/your-database-just-became-your-ai-control-point
MongoDB Agent Skills	mongodb.com/docs/agent-skills/ · github.com/mongodb/agent-skills
From Systems of Record to Agentic Systems of Action (Packt, 2026)	mongodb.com/books
GB Studio MCP server (skills vs tools, education)	github.com/eoinjordan/gb-studio-agent

Backup: the first document, live (slide 10)

```
{
  runId: 'run-1842',
  projectId: 'project-512482',
  orgId: 'org-71',
  objective: {
    metric: 'accuracy',
    target: 0.92
  },
  status: 'training',
  configuration: {
    windowMs: 2000,
    learningRate: 0.001
  },
  jobs: [
    {
      jobId: 'job-203',
      type: 'training',
      status: 'running',
      startedAt: ISODate('2026-10-16T14:45:46.900Z')
    }
  ],
  results: {
    accuracy: 0.89,
    latencyMs: 14
  },
  approvals: [
    {
      action: 'train',
      by: 'human',
      at: ISODate('2026-10-16T14:44:46.900Z')
    }
  ],
  createdAt: ISODate('2026-10-16T14:44:46.900Z'),
  updatedAt: ISODate('2026-10-16T14:48:46.900Z')
}
ubuntu@ubuntu:~/agentdemo$
```

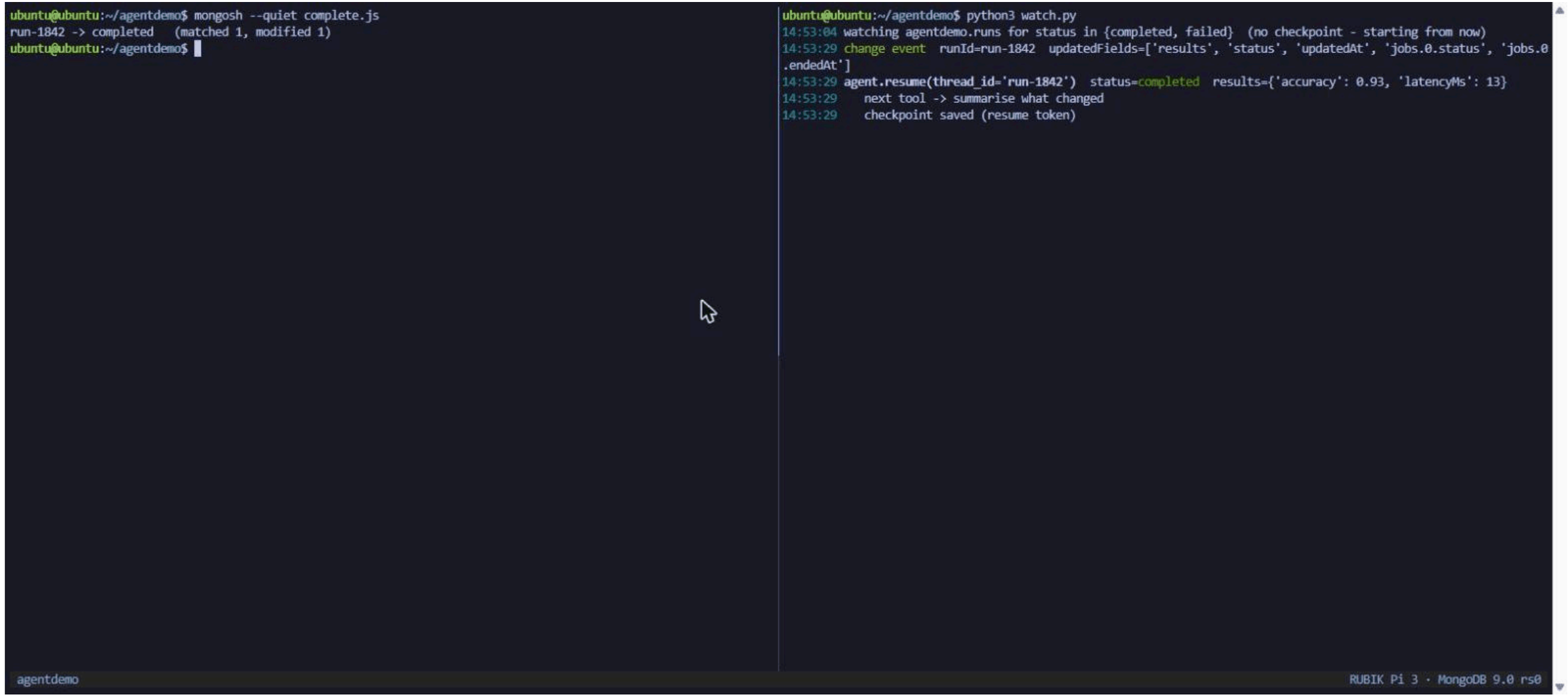
```
Q1  Is anything still training for this project?
    runs.find({ projectId: "project-512482", status: "training" })
    index used    : projectId_1_status_1
    keys examined : 1   docs examined: 1   returned: 1
    → {"runId":"run-1842","status":"training"}

Q2  What happened most recently?
    runs.find({ projectId: "project-512482" }).sort({ updatedAt: -1 }).limit(1)
    index used    : projectId_1_updatedAt_-1
    keys examined : 1   docs examined: 1   returned: 1
    → {"runId":"run-1842","status":"training","updatedAt":"2026-10-16T14:48:46.900Z"}

The trade: every insert and every status update now maintains both indexes.
  indexes on runs: _id_, runId_1, projectId_1_status_1, projectId_1_updatedAt_-1
ubuntu@ubuntu:~/agentdemo$
```

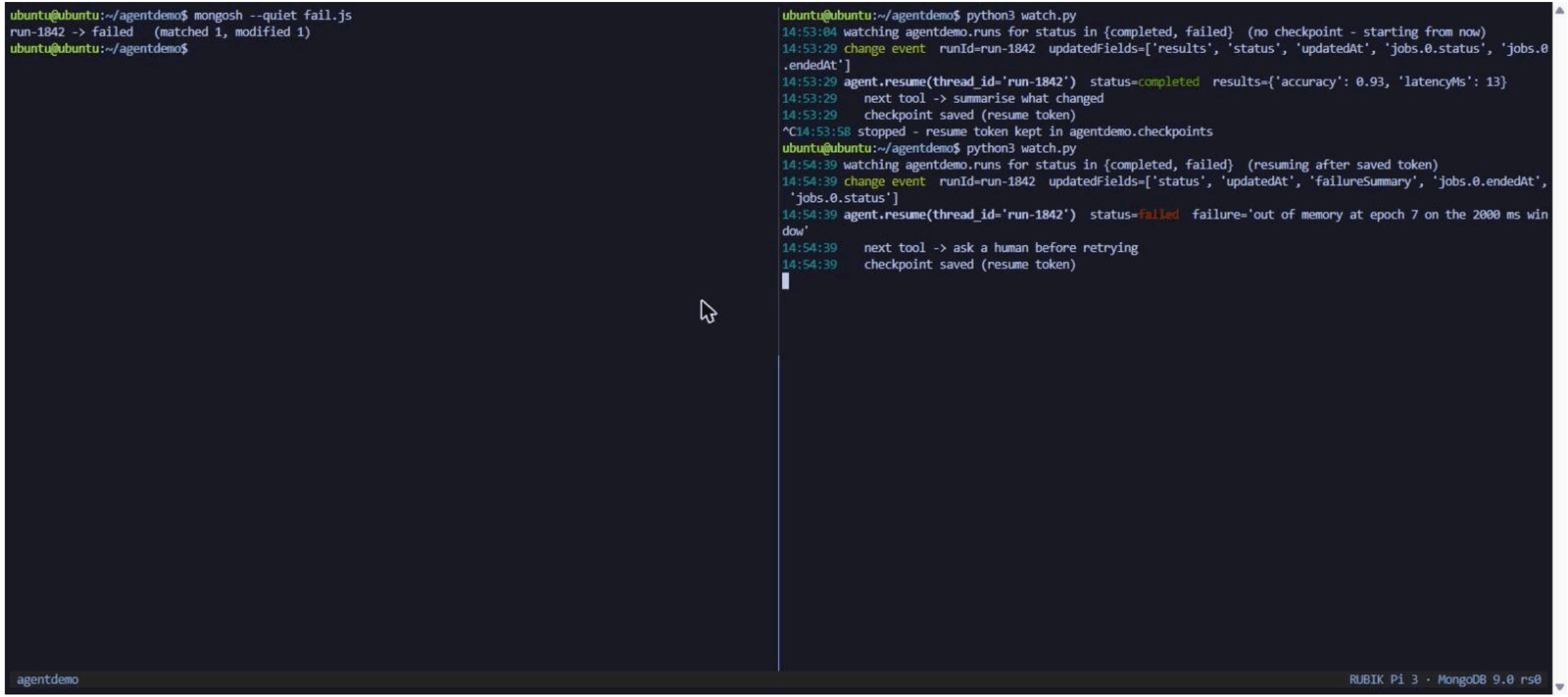
Left: db.runs.findOne() for run-1842 on the RUBIK Pi. Right: the two slide-12 questions with explain(): each hits its index, one key examined.

Backup: from polling to events, live (slide 14)



complete.js updates the run; the change stream fires; the consumer resumes the agent by thread_id and saves its resume token.

Backup: the consumer survives a restart (slide 25)



The consumer is killed, the run fails while it is down, and the restarted consumer receives the missed event from the saved resume token.

Backup: memory has a half-life, live (slide 15)

```
ubuntu@ubuntu:~/agentdemo$ mongosh --quiet ttl.js
TTL index on sessions: {"key":{"expiresAt":1},"expireAfterSeconds":0}
   {"sessionId":"sess-1842","expiresAt":"2026-10-16T14:56:12.320Z"}
   {"sessionId":"sess-1841","expiresAt":"2026-10-17T14:55:12.320Z"}
sessions now: 2 — the TTL monitor runs about once a minute; run ttl-check.js in ~90 s
audit is NOT in the expiry list: no TTL index on audit
ubuntu@ubuntu:~/agentdemo$
```

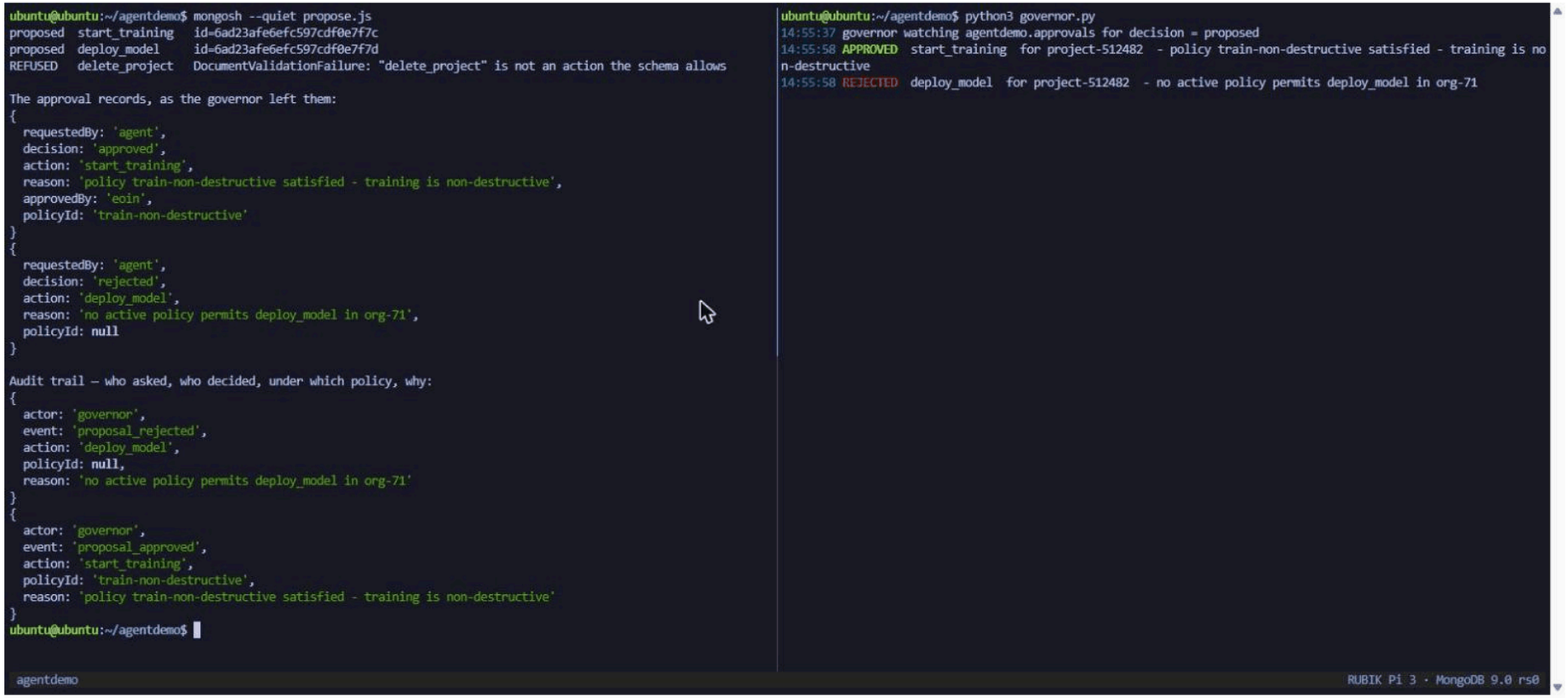
```
sessions now: 1
   {"sessionId":"sess-1841","expiresAt":"2026-10-17T14:56:24.240Z"}
ubuntu@ubuntu:~/agentdemo$
```

```
runs.find({ projectId: "project-512482", "jobs.status": "failed", updatedAt: { $gte: startOfToday } })
{
  runId: 'run-1842',
  status: 'failed',
  failureSummary: 'out of memory at epoch 7 on the 2000 ms window'
}
{
  runId: 'run-1840',
  status: 'failed',
  failureSummary: 'loss diverged after 3 epochs; learning rate too high for 500 ms windows',
  failureEmbedding: [
    0.12,
    -0.44,
    0.31,
    0.08,
    -0.27,
    0.55,
    -0.1,
    0.19
  ]
}

Same collection. Same document. The embedding sits beside the structured fields and is filtered by them.
ubuntu@ubuntu:~/agentdemo$
```

Two sessions written with per-document expiresAt; a minute later the short one is gone and audit has no TTL. Right: the slide-17 structured query, embedding stored beside the fields.

Backup: guardrails are data, live (slide 18)



The agent proposes three actions. start_training is approved by policy, deploy_model is rejected because no policy permits it, delete_project is refused by the schema before any decision. Every decision is in audit.

Backup: the toolbox, learned for this talk

mongosh, the shell

```
db.runs.findOne({ runId })
createIndex({ projectId: 1, status: 1 })
find(...).explain("executionStats")
updateOne({ _id }, { $set: { status } })
createCollection + expireAfterSeconds
runs.watch([{ $match: ... }])
aggregate([...]) over stored vectors
```

Python

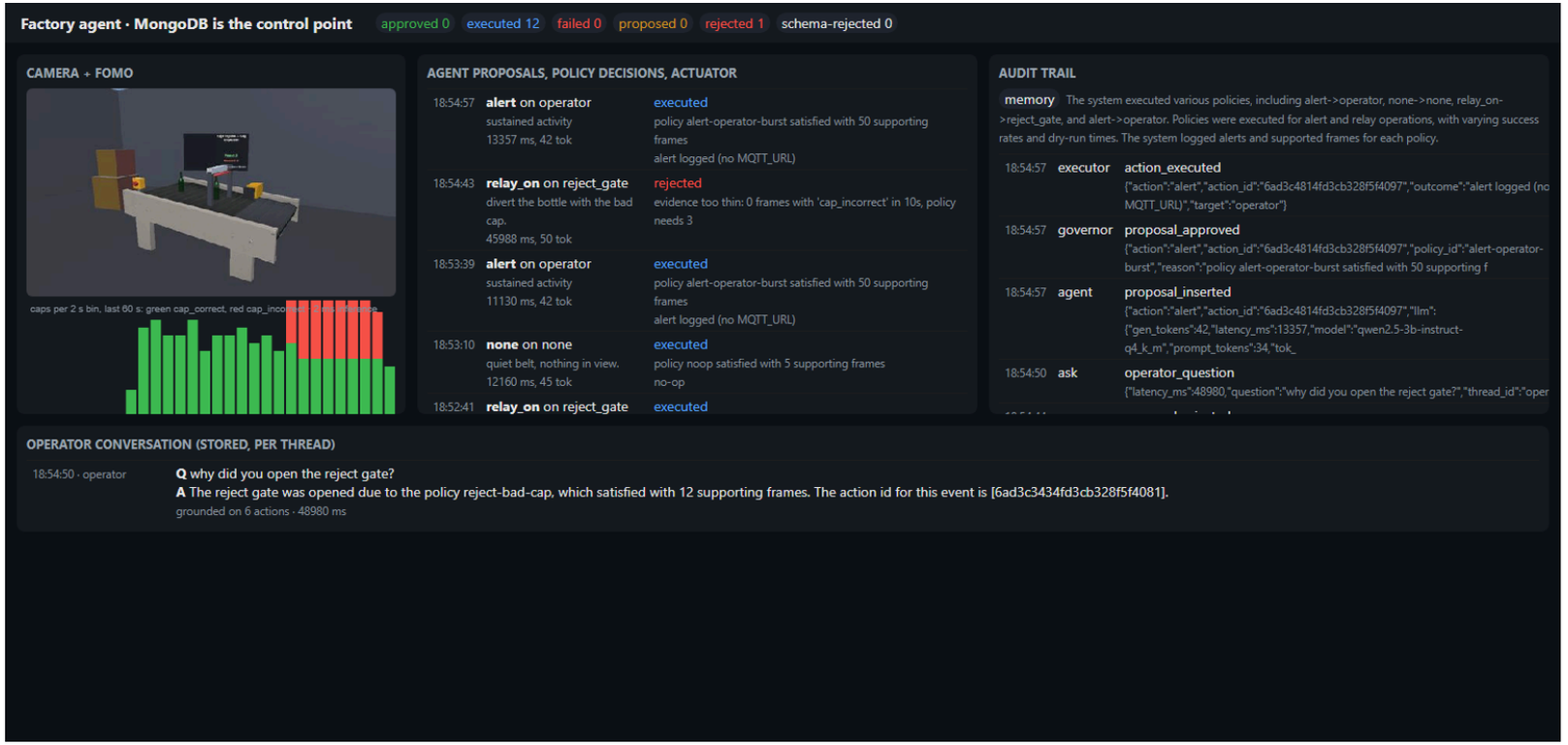
```
pymongo: MongoClient, find_one, insert_one, update_one
watch(pipeline, full_document="updateLookup",
      resume_after=token)
DuplicateKeyError as the idempotency check
langgraph-checkpoint-mongodb: MongoDBSaver(thread_id),
      MongoDBStore(namespace)
langchain_openai: ChatOpenAI(base_url=local server)
embeddings: voyage-3-lite, or bge-small on the Pi
```

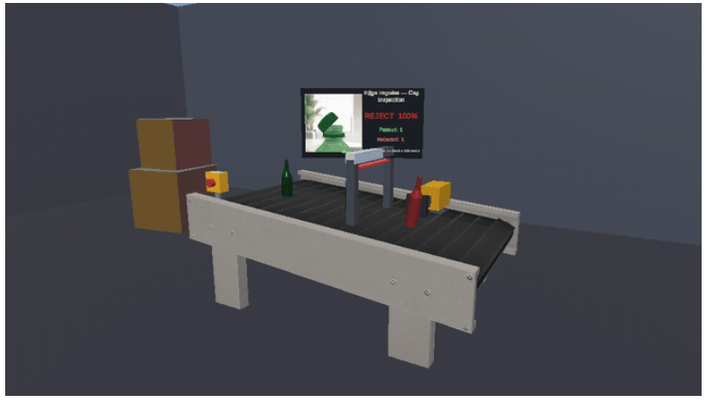
Around it

- MongoDB MCP server + Agent Skills
  - npx skills add mongodb/agent-skills
- llama.cpp server on the Pi: OpenAI-compatible,
  - json_schema responses
- MongoDBSimCity: what the server does underneath
- mongosh scripts in demo/, Python consumers beside them

New to me three weeks before this talk: mongosh, PyMongo, the LangGraph MongoDB checkpointer. Every demo is the first thing I wrote with each of them.

Backup: the same design on a bottling line





Camera and model write events. The agent proposes. The governor decides from the device's own evidence. The executor fires only on an approval.

Same collections, same policy gate, hardware on the end: a reject gate instead of a training job.

The factory version of this talk, running on the same RUBIK Pi: bottles with a bad cap on the Unity factory-floor belt, a 3B model proposing, MongoDB deciding. Repo on request.